

TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering
By**

P.DHEERAJ REDDY (VTU 26643)

*10211CS224
Full Stack Application Development
Slot : E1-B04*



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Ticket Booking of an Internal Department Event" by P.Dheeraj Reddy (VTU26643) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr. HAFIZUR RAHMAN

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of the Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

P.Dheeraj Reddy

Date:06/05/2026

ABSTRACT

The React JS based Ticket Booking System for Internal Department Events is a modern web application designed to simplify and manage the process of booking tickets for departmental events. It provides a centralized platform where administrators and users can interact easily. Administrators can create and manage events, update ticket availability, and monitor bookings, while users can view event details and book tickets online without difficulty. The system includes features such as user authentication, real-time updates, and booking confirmation to improve user experience. By reducing manual work and paperwork, it ensures better accuracy, transparency, and efficient management of event registrations, while also helping in tracking participation and organizing successful events.

Keywords: React.js, Node.js, MySQL, Web Application, Ticket Booking System, Internal Department Events, Event Registration, User Management, Database System, Online Booking, Event Scheduling.

LIST OF FIGURES

1.1	Frame work Architecture	3
3.1	Login Page	9
3.2	Booking Page	12
4.1	System Architecture of Ticket Booking Web Application	13
5.1	Home Page	18

LIST OF TABLES

5.1	Application Comparison	17
7.1	Mapping of the Project to Complex Engineering Problem (CEP)	20
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	21
7.3	SDG Mapping for the Project	21

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JSX	JavaScript XML
JWT	JSON Web Token
MySQL	My Structured Query Language
NPM	Node Package Manager
ORM	Object-Relational Mapping
QR	Quick Response Code
UPI	Unified Payments Interface

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5

3.2	Backend Implementation	7
3.3	Database Implementation	11
4	METHODOLOGIES	13
4.1	Project Architecture	13
4.2	DataFlow Diagram	14
4.3	Module 1: User Management	15
4.4	Module 2: Event Management	15
4.5	Module 3: Registration and Notification	15
5	RESULTS AND DISCUSSIONS	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	19
6.1	Conclusion	19
6.2	Future Enhancements	19
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	20
7.1	Mapping of the Project to Complex Engineering Problem	20
7.2	Mapping of the Project to Complex Engineering Ac- tivities (CEA)	21
7.3	SDG Mapping (CEP Section)	21
8	REFERENCES	22

Chapter 1

INTRODUCTION

1.1 Introduction

The React JS–based ticket booking web app simplifies managing internal department events. Organizers can create events, and users can view details and book tickets easily. Real-time updates and confirmations enhance user experience while reducing manual work and improving accuracy.

1.2 Aim of the Project

The aim of this project is to develop a web application using React JS for booking tickets to internal department events in a simple and efficient way. It enables users to easily view event details and book tickets online without any difficulty. The system reduces manual work and minimizes errors in event management. Overall, it provides a fast, reliable, and user-friendly platform for handling event ticket bookings.

1.3 Scope of the Project

The scope of this project is to build an easy-to-use web application for booking tickets to internal department events. It helps organizers create events, manage tickets, and keep track of bookings without much effort. Users can simply check event details and book their tickets online in a quick and convenient way. In the future, the system can be improved by adding features like online payments, notifications, and reports.

1.4 Framework

The EduTickets system is built using a modern web framework that combines both frontend and backend technologies. The frontend is developed with React JS, making the interface simple, interactive, and responsive with the help of CSS. The backend uses Node.js and Express.js to handle server operations and manage data smoothly. Important features like email verification and password security are added using Nodemailer and bcryptjs. JWT is used to keep user login sessions safe and secure. All the data is stored in a MySQL database using MySQL Workbench, and the system follows a three-tier structure for better performance and easy scaling.

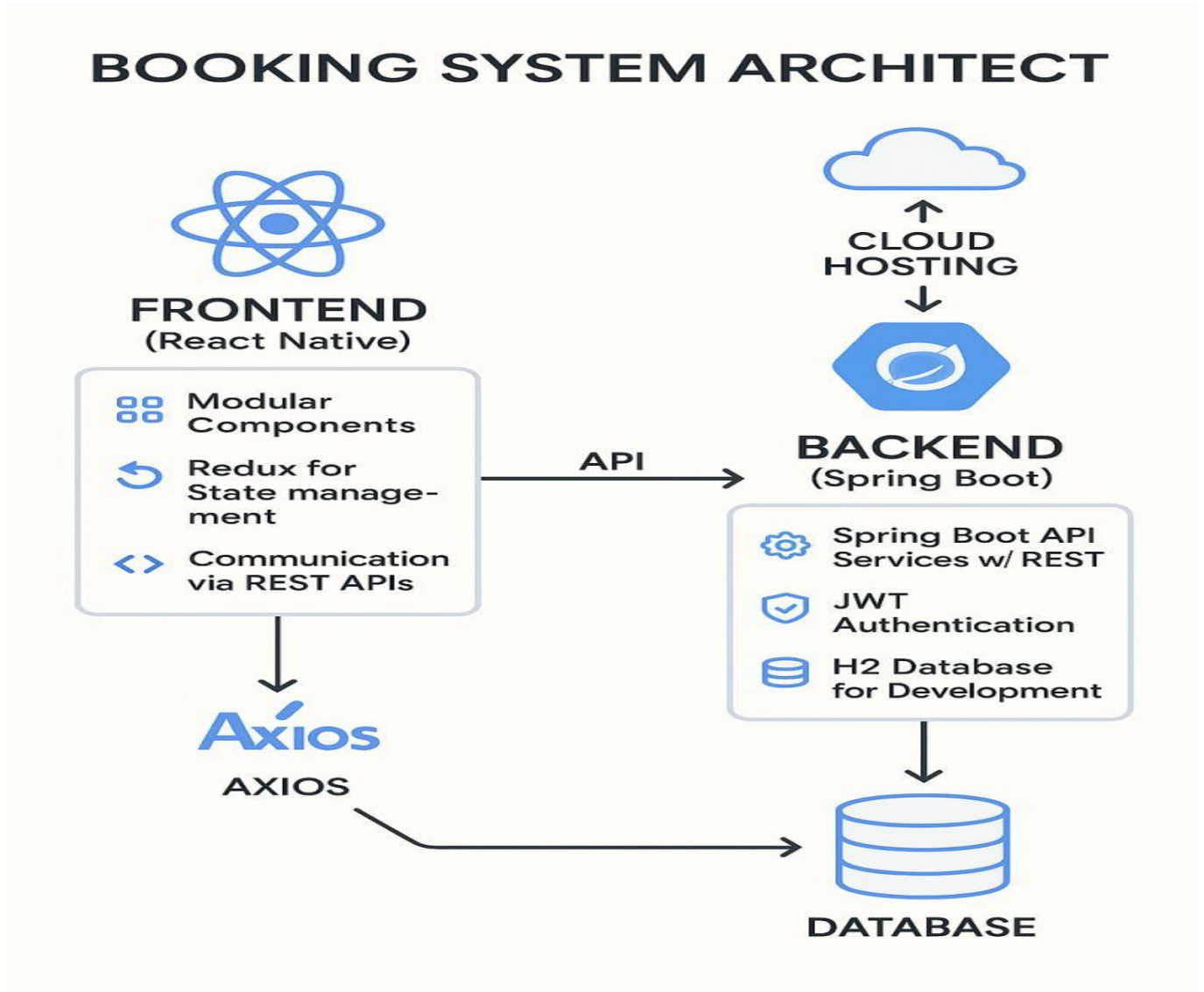


Figure 1.1: Frame work Architecture

[Figure 1.1] illustrates the overall architecture of the booking system, integrating a React Native frontend with a Spring Boot backend through REST APIs. It demonstrates the data flow between client, server, and database, emphasizing modular design, secure authentication, and cloud-based deployment.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Reddy and Kumar developed a web-based event management system using modern frontend and backend technologies to streamline event registration and booking processes in educational institutions [8]. Singh and Verma proposed a cloud-integrated ticket booking system that improves scalability and ensures high availability for large-scale events [9].

Sharma et al. designed a secure REST API-based architecture using Spring Boot to enable efficient communication between client and server components in real-time applications [10]. Gupta et al. introduced a mobile-based event booking application with personalized recommendations and notification systems to enhance user engagement [11].

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

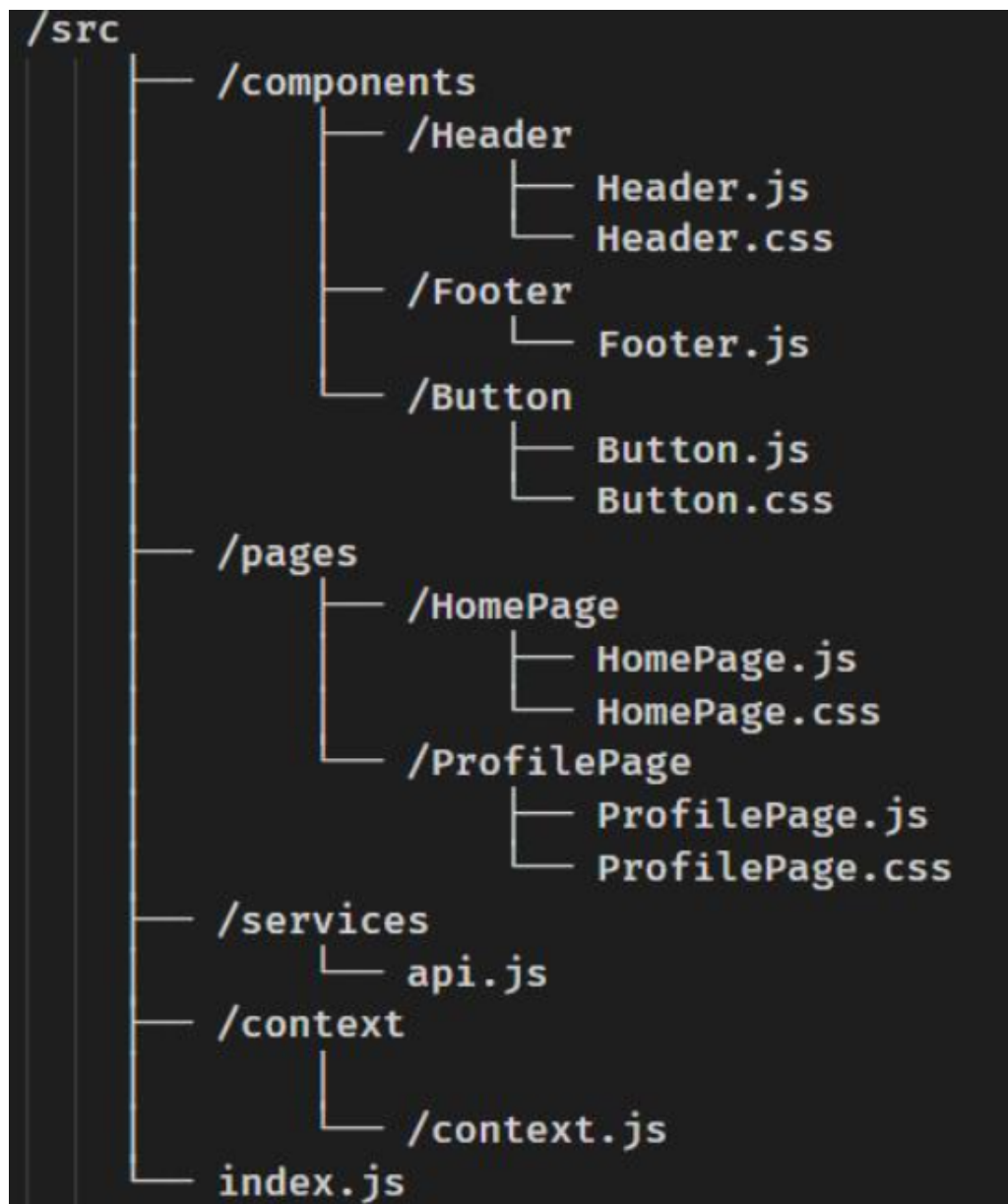
Environment Setup

The frontend for the ticket booking application is developed using React JS, which enables the creation of dynamic and reusable user interface components. Visual Studio Code is used as the primary code editor due to its lightweight nature, extensive extensions, and debugging support.

A modern web browser such as Google Chrome is used for testing and debugging the application. Developer tools available in the browser help in inspecting elements, monitoring network requests, and identifying performance issues.

To set up the development environment, Node.js and npm (Node Package Manager) are installed. These tools are essential for managing dependencies and running the React application.

3.1.2 Structure of the Project



The frontend structure is organized using reusable React components, ensuring modularity and easy maintenance. It separates pages, components, styles, and assets to provide clear organization and smooth navigation. This design improves scalability and enhances the overall user interface performance.

3.1.3 Execution Procedure

The frontend application is executed using a local development server provided by React. By running commands like `npm start`, the project launches in a browser through a local URL. This allows real-time updates and efficient testing during development. Users can interact with the interface to explore events and book tickets easily.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Node.js and Express.js to handle server-side operations efficiently. These technologies enable the creation of scalable and high-performance web applications. Visual Studio Code is used as the development environment due to its rich ecosystem of extensions and debugging capabilities.

`npm` (Node Package Manager) is used for managing project dependencies and installing required libraries. The backend project is initialized using `npm init` and structured to follow best practices for maintainability.

3.2.2 Structure of the Project

```
├── /config
│   └── database.js      # Database configuration
├── /controllers
│   ├── productController.js # logic for products
│   └── userController.js    # logic for users
├── /models
│   ├── productModel.js     # Mongoose schema or SQL schema for products
│   └── userModel.js        # Mongoose schema or SQL schema for users
├── /routes
│   ├── productRoutes.js    # Routes for product-related requests
│   └── userRoutes.js        # Routes for user-related requests
├── /middleware
│   ├── authMiddleware.js   # Middleware for authentication
│   └── errorHandler.js     # Custom error-handling middleware
├── /utils
│   └── helpers.js          # Utility functions used across the app
├── /services
│   ├── emailService.js     # Service for sending emails
│   └── paymentService.js    # Service for handling payments
├── /logs
│   └── app.log             # Log files for error tracking
├── /public
│   └── uploads             # Directory for file uploads
├── app.js                 # Entry point for the Node.js server
├── package.json           # Project dependencies and scripts
└── README.md              # Project overview and documentation
```

The backend structure follows the MVC (Model-View-Controller) architecture, ensuring a clear separation of concerns between different components. Controllers handle request processing, models manage data and database interactions, while routes define application endpoints. This organized structure improves maintainability, scalability, and overall system performance.

3.2.3 Execution Procedure

The backend server is started using commands like `node app.js` or `npm start`. Once the server runs, it connects to the MySQL database and begins handling client requests. APIs are used to communicate between frontend and backend seamlessly. This ensures smooth functionality for user authentication, event management, and ticket booking.

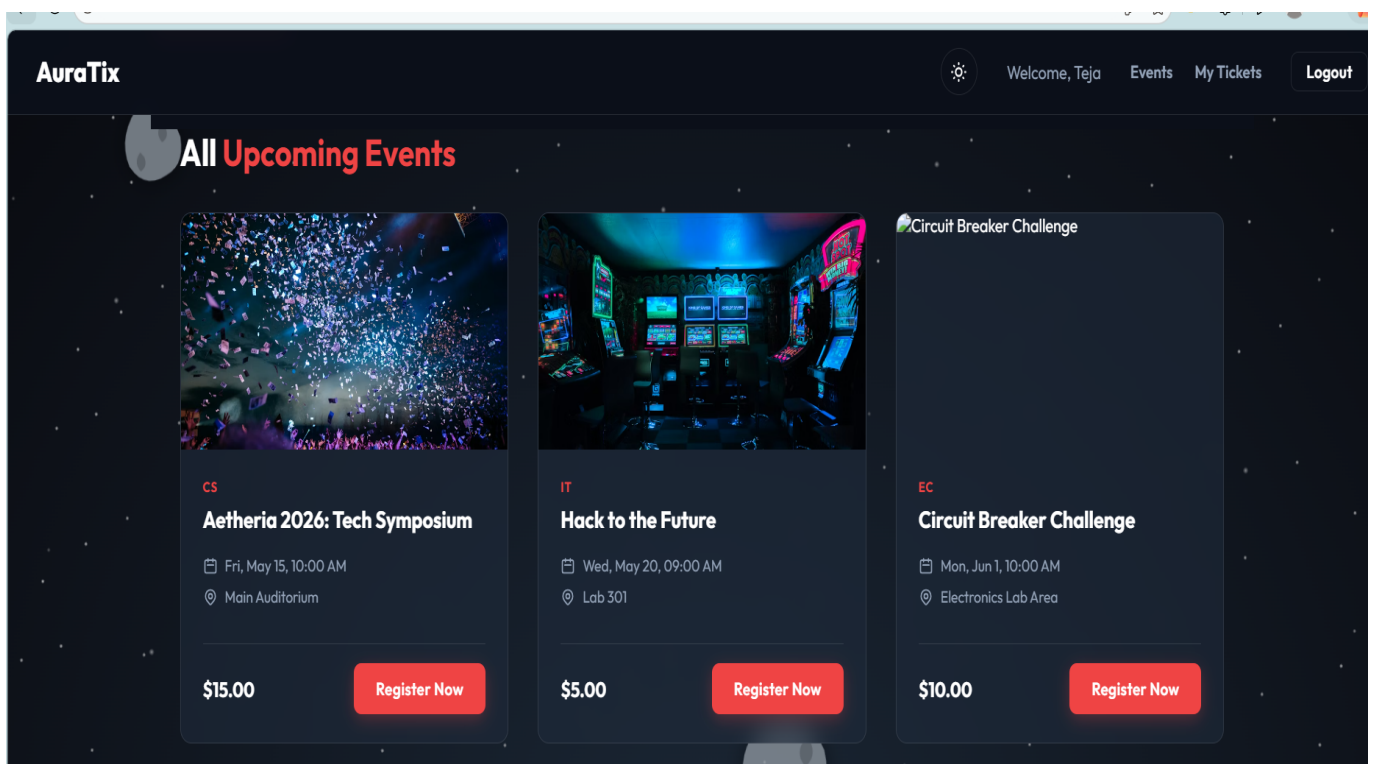


Figure 3.1: Login Page

[Figure 3.3] shows the login page provides a secure interface for users to access the AuraTix Event Booking System using their credentials. It validates user input and authenticates access using backend APIs and JWT-based security mechanisms. Upon successful login.

3.3 Database Implementation

3.3.1 Database Configuration

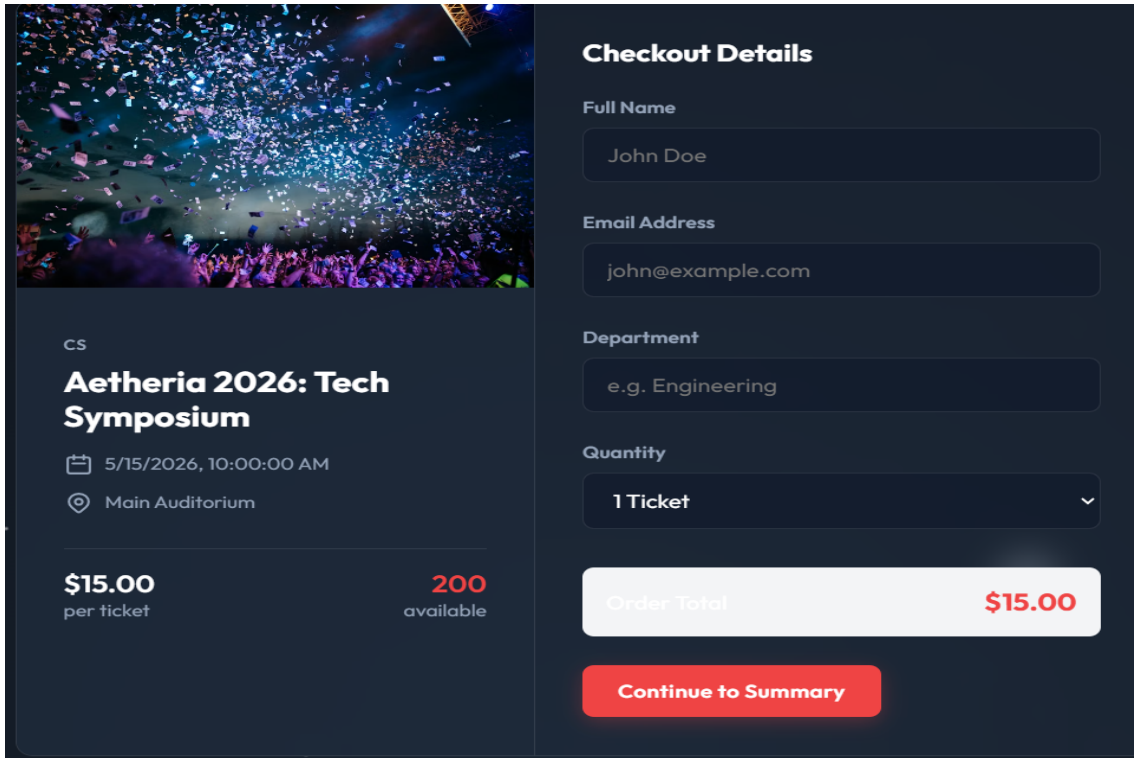
The application uses MySQL as the database system for storing all essential data. It is configured using tools like MySQL Workbench for easy management. The backend connects to the database using appropriate drivers to perform operations. This setup ensures secure, reliable, and efficient data handling.

3.3.2 Schema Structure

The database schema is designed to organize data into structured and related tables. It includes entities like users, events, bookings, and notifications. Relationships are maintained using primary and foreign keys to ensure data consistency. This design helps reduce redundancy and improves overall system performance.

3.3.3 Tables Created

The system includes key tables such as Users, Events, Bookings, and Notifications. Each table stores specific information like user details, event data, and ticket records. Important fields include IDs, names, dates, and status indicators for proper tracking. These tables work together to support smooth ticket booking and event management.



The image shows a booking page for the 'Aetheria 2026: Tech Symposium'. The left side features a large image of a concert stage with confetti, followed by event details: 'CS', 'Aetheria 2026: Tech Symposium', '5/15/2026, 10:00:00 AM', and 'Main Auditorium'. Below this, the price is '\$15.00 per ticket' and '200 available'. The right side is titled 'Checkout Details' and contains form fields for 'Full Name' (John Doe), 'Email Address' (john@example.com), and 'Department' (e.g. Engineering). A 'Quantity' dropdown is set to '1 Ticket'. At the bottom right, a white box shows 'Order Total' as '\$15.00', and a red button says 'Continue to Summary'.

Event Details	
CS	Aetheria 2026: Tech Symposium
📅	5/15/2026, 10:00:00 AM
📍	Main Auditorium
\$15.00	200
per ticket	available

Checkout Details	
Full Name	John Doe
Email Address	john@example.com
Department	e.g. Engineering
Quantity	1 Ticket
Order Total	\$15.00
Continue to Summary	

Figure 3.2: Booking Page

[Figure 3.4] shows the booking page allows users to enter required details such as name, email, and ticket quantity to reserve event tickets. It displays event information and calculates the total cost dynamically based on the number of tickets selected. Upon submission, the booking details are processed and a confirmation is generated for the user.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The ticket booking web application follows a three-tier architecture consisting of frontend, backend, and database layers. The frontend is developed using React JS to provide a smooth and interactive user experience.

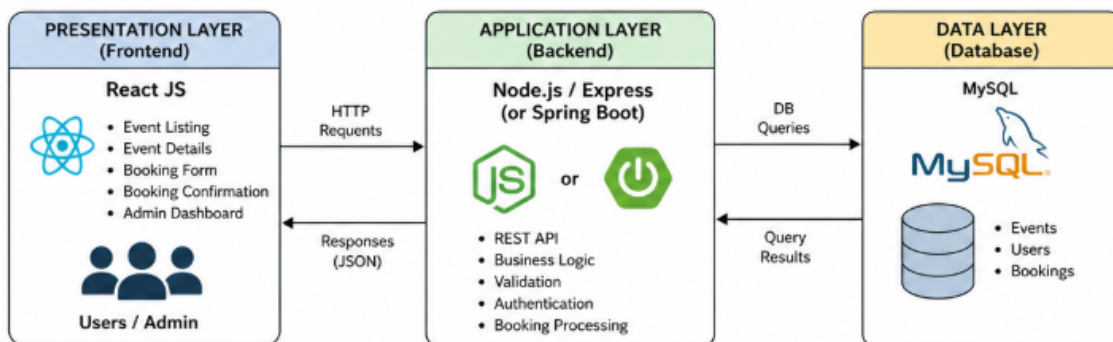
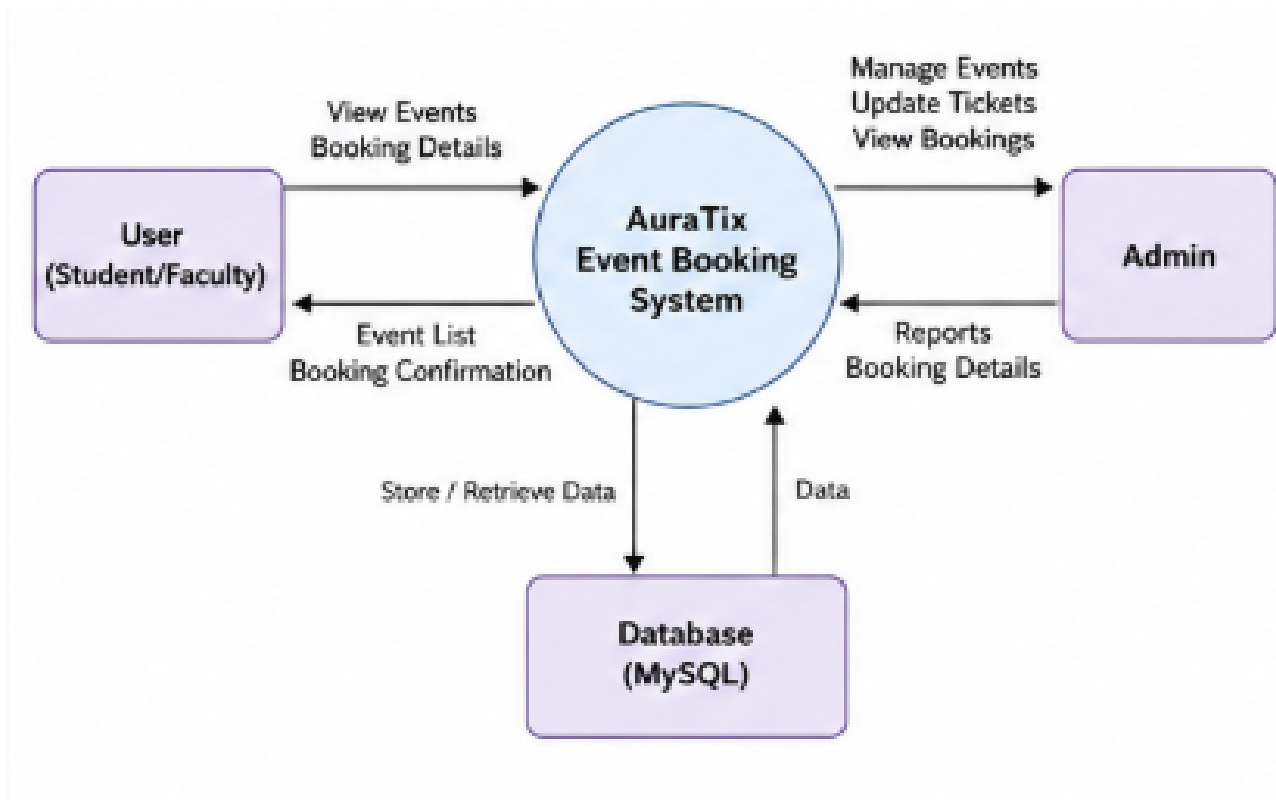


Figure 4.1: System Architecture of Ticket Booking Web Application

Figure 4.1 illustrates the overall architecture of the booking system, integrating a React Native frontend with a Spring Boot backend through REST APIs.

4.2 DataFlow Diagram

The Data Flow Diagram (DFD) shows how data moves in the ticket booking system. Admins and students provide inputs like event details and bookings, which are processed and stored in the database. The system then outputs confirmations, event listings, and notifications.



[[Figure 4.2] shows the Data Flow Diagram (DFD) represents how data moves within the ticket booking system, including inputs from users and admins. It illustrates the processing of bookings and event details, along with outputs such as confirmations, event listings, and notifications.

4.3 Module 1: User Management

The User Management module handles all user-related operations in the system. It includes features like user registration, login, and profile management. Different roles such as admin and student are managed with proper access control. Security is maintained using password encryption and JWT-based authentication. This ensures that only authorized users can access specific features. It also helps in maintaining user data in a secure and organized way.

4.4 Module 2: Event Management

The Event Management module allows admins to create and manage internal department events efficiently. It includes adding event details such as title, date, time, venue, and description. Users can view available events and choose tickets for booking. The module also helps in updating or deleting events when required. It ensures smooth handling and organization of all event-related activities. This improves the overall experience for both organizers and participants.

4.5 Module 3: Registration and Notification

This module manages ticket booking and user communication within the system. Students can easily register and book tickets through an

online interface. After booking, users receive instant confirmation messages. Notifications are also sent for event reminders and important updates. This keeps users informed about their bookings and upcoming events. It enhances communication and improves user engagement in the system.

Advantages of this Project

a) Easy Ticket Management: The system makes it simple to create, update, and manage events and ticket bookings, reducing manual effort.

b)Efficient Event Management: Organizers can easily create, update, and manage events and track bookings in one place.

c)Time Saving and Accurate: It reduces manual work, minimizes errors, and ensures accurate handling of event and booking data. **d)Centralized Data Management:**All event and booking information is stored in one place, making it easy to access and manage. **e)Improved User Experience:**Users can easily view event details and book tickets quickly with a simple and user-friendly interface.

Disadvantages

1. May face technical issues like server downtime or slow performance
2. Requires regular maintenance for security and performance

Chapter 5

RESULTS AND DISCUSSIONS

The React JS–based Ticket Booking System was successfully implemented and tested. It enables efficient event management and ticket booking, including event creation and confirmations. The system reduces manual work, improves communication, and allows organizers and users to interact smoothly while maintaining accurate data in a centralized database.

Table 5.1: Application Comparison

Applications	Advantages
Eventbrite	Provides online ticket booking and event management features
Booking System	Supports ticket booking with a user-friendly interface
Ticket Tailor	Simple platform for creating and managing ticket sales
Proposed System	Cost-effective and easy to use for internal department events

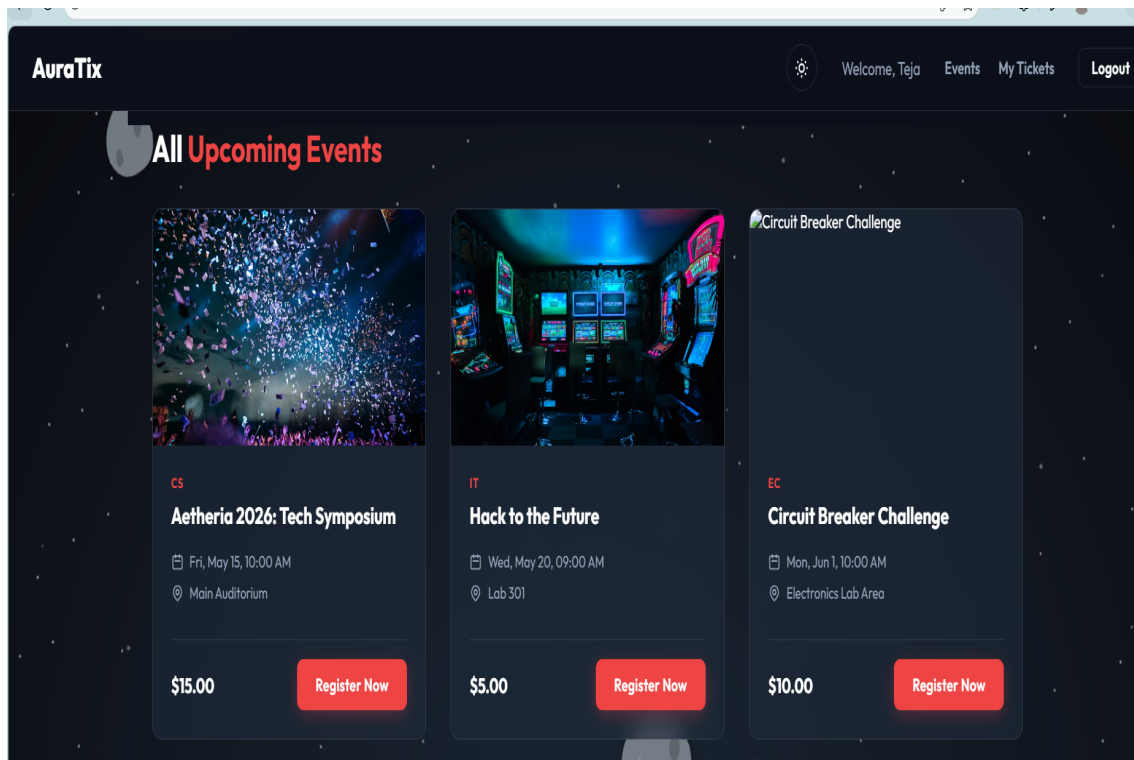


Figure 5.1: Home Page

[Figure 5.1] shows the Home Page provides an intuitive interface for users to explore events, view detailed information, and initiate bookings . It serves as the central hub of the application, offering easy navigation, featured events, and quick access to essential functionalities. Users can search, filter, and browse events based on categories, dates, and preferences for a personalized experience.

The page also displays notifications, upcoming bookings, and user-specific recommendations to enhance usability and engagement.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System efficiently manages campus events through a centralized platform. It simplifies event creation, registration, and communication, reduces manual work, and improves accuracy, coordination, and student participation.

6.2 Future Enhancements

The system can be enhanced by adding a mobile app for better access. Features like QR-based attendance, real-time analytics, and personalized recommendations can be included. Integration with email and calendar services, along with improved security and performance, will make the system more reliable and scalable.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	In-depth knowledge	WK3, WK4, WK5, WK6	Uses React, backend, and database.
WP2	Conflicting Requirements	Trade-offs	WK4, WK5, WK6	Balances performance and scalability.
WP3	Analysis Required	Original analysis	WK3, WK4, WK5	Booking logic and validation.
WP4	Familiarity of Issues	Less common issues	WK3, WK5	Real-time updates and concurrency.
WP5	Applicable Codes	Beyond standards	WK5, WK6	API and system integration.
WP6	Stakeholders	Multiple users	WK6	Organizers and students.
WP7	Interdependence	Dependent modules	WK5, WK6	Frontend, backend, database.

Table 7.1: Mapping of the Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events
SDG 9	Innovation	Promotes React-based systems
SDG 10	Reduced Inequalities	Accessible to all users
SDG 11	Sustainable Communities	Encourages organized events

Table 7.3: SDG Mapping for the Project

References

- [1] Eventbrite, "Event Management and Ticketing Platform." [Online]. Available: <https://www.eventbrite.com>
- [2] Whova, "Event Management Software." [Online]. Available: <https://www.whova.com>
- [3] Ticketbud, "Online Event Ticketing Platform." [Online]. Available: <https://www.ticketbud.com>
- [4] Humanitix, "Event Ticketing Platform." [Online]. Available: <https://www.humanitix.com>
- [5] Cvent, "Event Management Software." [Online]. Available: <https://www.cvent.com>
- [6] DICE, "Mobile Ticket Booking Platform." [Online]. Available: <https://dice.fm>
- [7] Ticket Tailor, "Event Ticketing and Booking System." [Online]. Available: <https://www.tickettailor.com>